accenture

GitHub and Codebase Management

Platform Features, Repositories & Best Practices

Chapter Overview

What is GitHub?

Section 1

Introduction to GitHub

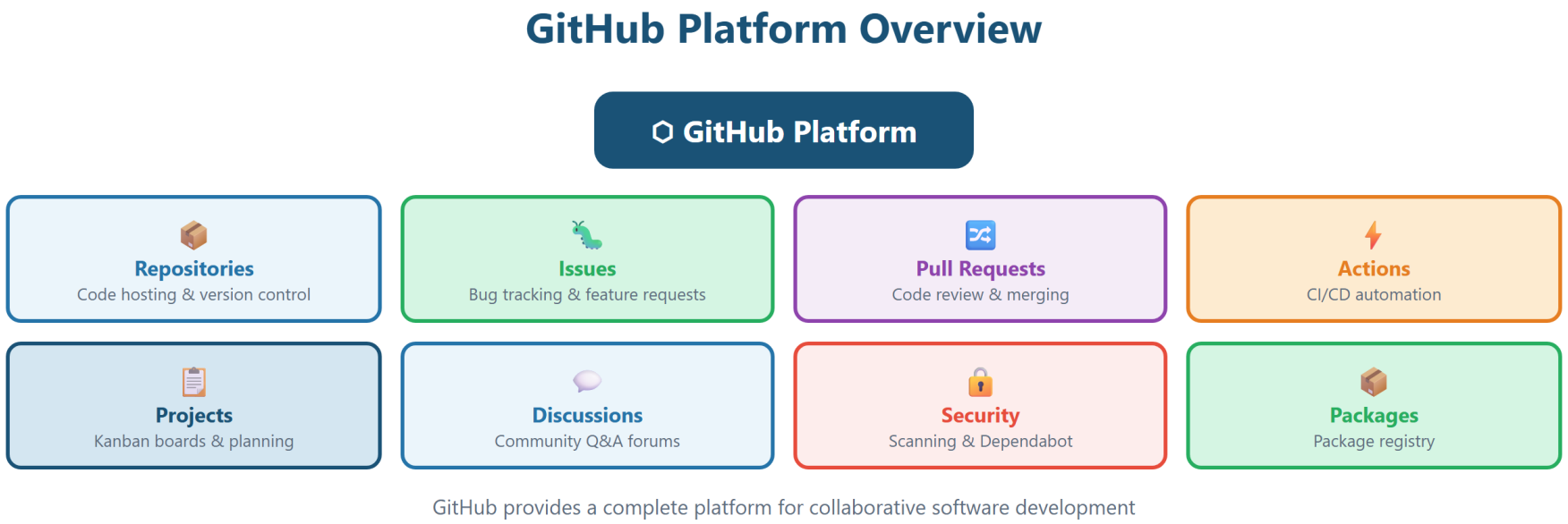
What is GitHub?

- A cloud-based platform for hosting Git repositories
- The world's largest developer community — 100M+ users
- Owned by Microsoft since 2018

Why GitHub?

- Provides a web interface on top of Git version control
- Adds collaboration tools: issues, pull requests, code review
- Hosts open-source and private projects alike
- Integrates CI/CD, project management, and security scanning

GitHub Platform Overview



GitHub vs Git

Git (The Tool)

- Distributed version control system — runs locally on your machine
- Tracks changes, creates commits, manages branches
- Works entirely offline — no internet connection required

GitHub (The Platform)

- Cloud hosting for Git repositories — your code lives online
- Adds collaboration features Git doesn't have (issues, PRs, Actions)
- Provides a web UI for browsing code, reviewing changes, and managing projects

GitHub Repository Example

What You'll Find in a Repo

- A repository (‘repo’) is a project’s home on GitHub
- Source code files organized in folders
- README.md — project description and setup instructions
- Commit history — every change ever made
- Branches — parallel versions of the codebase
- Releases — tagged versions ready for deployment

The GitHub Interface — Key Tabs

Code

- Browse files, view README, switch branches

Issues

- Track bugs, feature requests, and tasks

Pull Requests

- Propose, review, and merge code changes

Actions

- Automated workflows — CI/CD pipelines

Settings

- Repository configuration, access control, branch protection

GitHub Features & Collaboration

Section 2

Issues — Tracking Work

What are Issues?

- Built-in task tracker — report bugs, request features, plan work
- Each issue has a title, description, labels, assignees, and comments

Key Features

- Labels — categorize issues (bug, enhancement, help wanted)
- Milestones — group issues into release targets
- Assignees — assign responsibility to team members
- Templates — standardize bug reports and feature requests

Pull Requests — Code Review

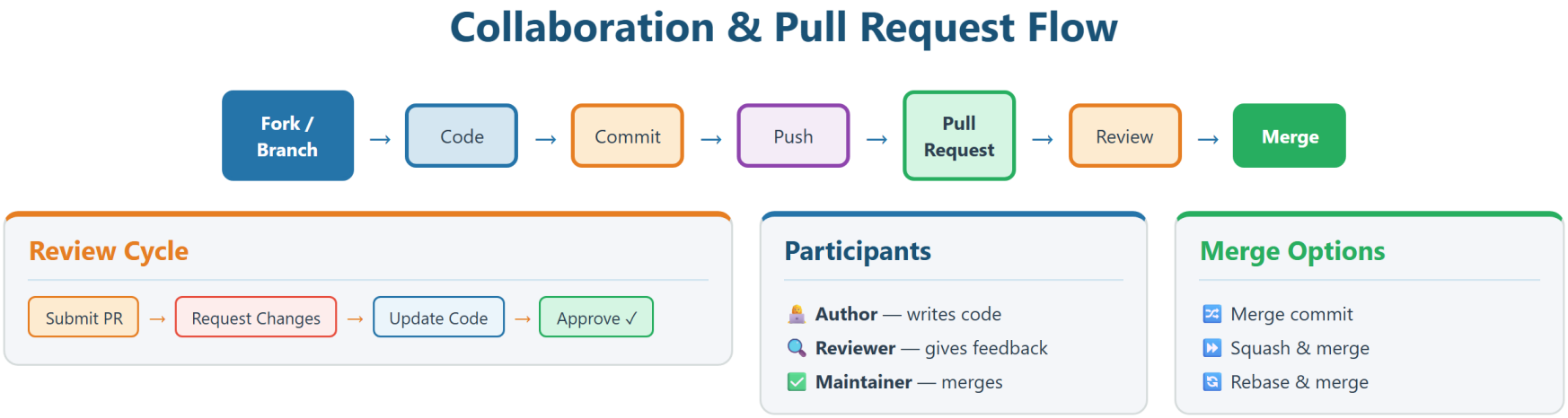
What is a Pull Request (PR)?

- A request to merge changes from one branch into another
- The primary mechanism for code review on GitHub

PR Workflow

- Create a branch → make changes → open a PR
- Reviewers examine the diff, leave comments, request changes
- Automated checks (CI/CD) run against the PR
- Once approved, merge the PR into the target branch

Collaboration Workflow



Code Review Best Practices

For Authors

- Keep PRs small and focused — one feature or fix per PR
- Write a clear description explaining what and why
- Link related issues using keywords (Fixes #42)

For Reviewers

- Review promptly — don't let PRs sit for days
- Focus on logic, security, and maintainability — not style nitpicks
- Use 'Suggest changes' for specific code improvements

Other GitHub Features

GitHub Actions

– Built-in CI/CD platform

GitHub Pages

– Free static site hosting directly from your repository

GitHub Discussions

– Forum-style conversations for Q&A and community engagement

GitHub Projects

– Kanban boards and tables for project planning and tracking

GitHub Codespaces

– Cloud-based development environments — VS Code in the browser

GitHub Actions

Section 3

What are GitHub Actions?

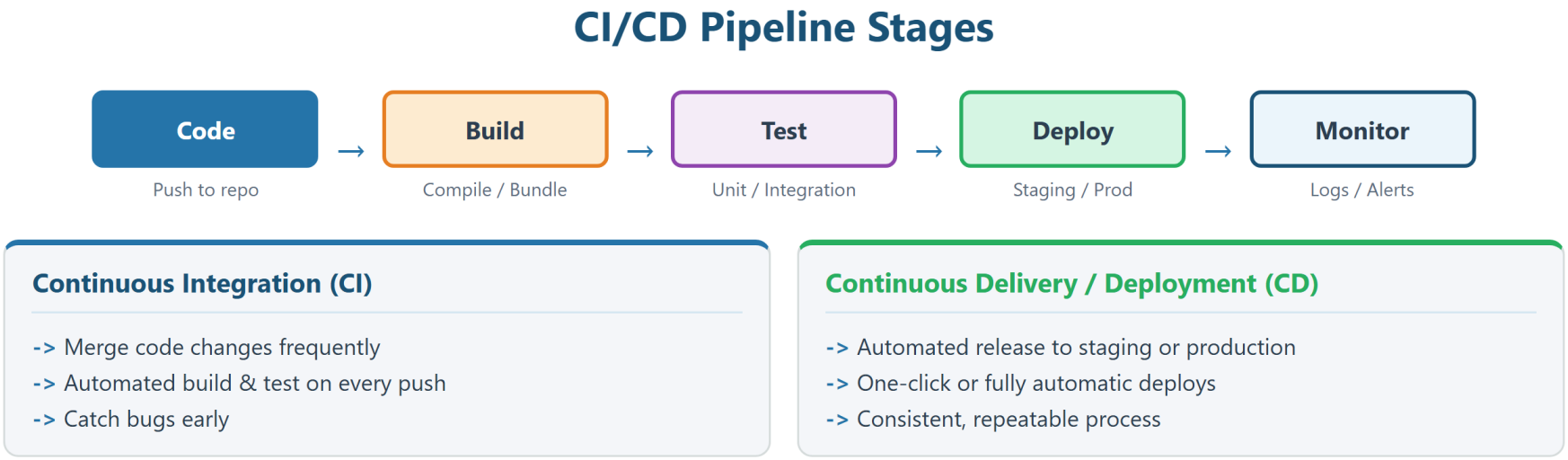
Built-in CI/CD Platform

- Automate software workflows directly in your repository
- No external CI server needed — runs on GitHub-hosted or self-hosted runners
- Free for public repos; generous free tier for private repos

What Can You Automate?

- Build, test, and deploy your code on every push
- Run linters, formatters, and security scanners automatically
- Publish packages, update documentation, send notifications
- Any scriptable task triggered by repository events

CI/CD Pipeline Stages



Workflow Anatomy

Workflow

- A YAML file in .github/workflows/ that defines an automated process
- Triggered by events (push, pull_request, schedule, manual)

Jobs

- A set of steps that execute on the same runner (virtual machine)
- Jobs run in parallel by default — use 'needs' for ordering

Steps

- Individual tasks within a job — run a command or use a reusable action
- Steps execute sequentially and share the same filesystem

Workflow, Jobs & Steps

Anatomy of a GitHub Actions Workflow

```yaml
name: CI Pipeline              ← workflow name
on:                                ← trigger events
  push:
    branches: [main]
  pull_request:
    branches: [main]
jobs:                               ← job definitions
  build:
    runs-on: ubuntu-latest       ← runner
    steps:                             ← step list
      - uses: actions/checkout@v4 ← action ref
      - run: npm install
      - run: npm test
```

Keys
YAML property names

Values
Configuration strings

Comments
← Annotations

File Location
`.github/workflows/ci.yml`

Workflow Triggers

Code Events

- push — run on every push to specified branches
- pull_request — run when a PR is opened, updated, or merged

Scheduled Triggers

- schedule — cron syntax for recurring jobs (e.g., nightly builds)

Manual & Other Triggers

- workflow_dispatch — run manually from the Actions tab
- release — triggered when a release is published
- repository_dispatch — triggered by external API calls

Actions Marketplace & Reusable Actions

What is the Marketplace?

- Library of 20,000+ pre-built actions shared by the community
- Use with 'uses:' keyword — e.g., actions/checkout@v4

Essential Actions

- actions/checkout — clone your repo into the runner
- actions/setup-node / setup-python / setup-java — install runtimes
- actions/cache — cache dependencies for faster builds
- actions/upload-artifact — save build outputs for later jobs

Environments, Secrets & Variables

Secrets

- Encrypted values stored in repository or organization settings
- Accessed via ${{ secrets.MY_SECRET }} — never printed in logs
- Use for API keys, credentials, tokens, and certificates

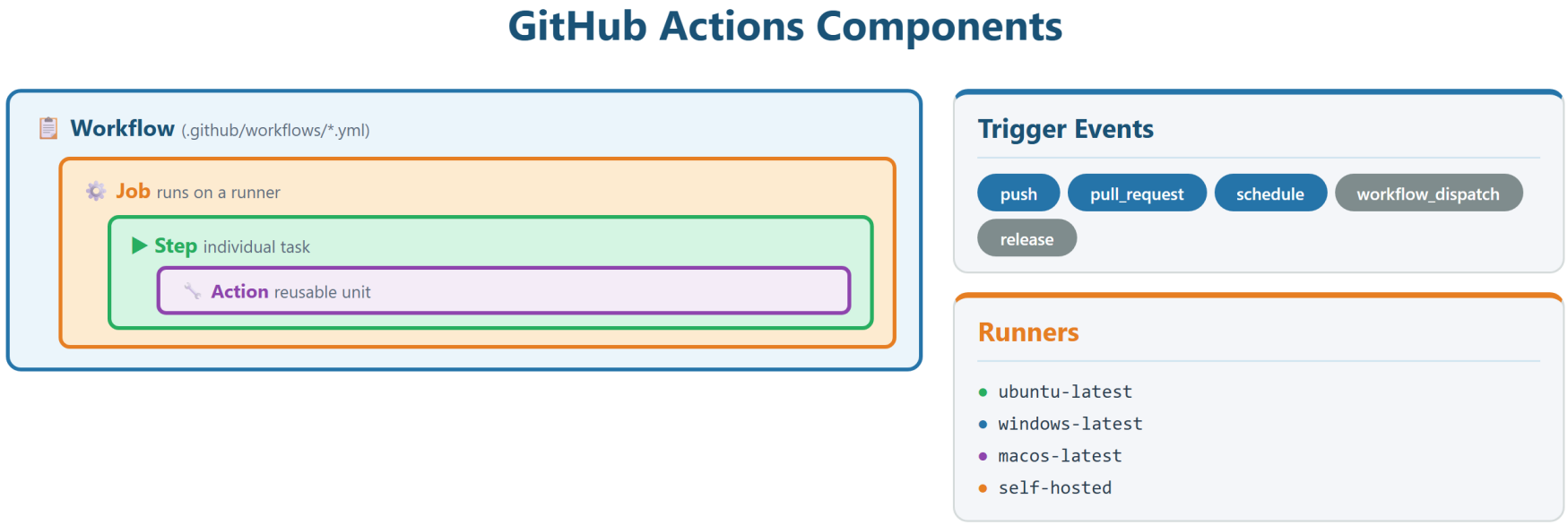
Environment Variables

- Set at workflow, job, or step level using 'env:' key
- Non-sensitive configuration: NODE_ENV, API_URL, LOG_LEVEL

Environments

- Named targets (staging, production) with their own secrets and protection rules
- Require manual approval before deploying to production

GitHub Actions Workflow Overview



Workflows automate build, test, and deployment — triggered by repository events

Codebase Management

Section 4

What is Codebase Management?

Definition

- The practices and tools used to organize, maintain, and evolve a software project's source code

Key Aspects

- Version control — track every change with full history
- Branching & merging — parallel development without conflicts
- Dependency management — track external libraries and packages
- Documentation — README, contributing guides, API docs
- Code quality — linting, formatting, testing, CI/CD

Codebase Management — Key Aspects

Version Control in Codebase Management

Why Version Control Matters

- Every change is recorded — who changed what, when, and why
- Roll back to any previous state if something breaks
- Multiple developers can work simultaneously without overwriting each other

GitHub Adds

- Central remote repository — single source of truth for the team
- Pull request workflow for safe, reviewed code integration
- Audit trail — every merge, review, and approval is logged

Branching Strategies

GitHub Flow (Recommended for Most Teams)

- main branch is always deployable
- Create feature branches for all changes
- Open a PR, get reviews, merge back to main

Common Branch Naming Conventions

- feature/add-login-page — new features
- bugfix/fix-header-alignment — bug fixes
- hotfix/security-patch — urgent production fixes

Dependency Management

What are Dependencies?

- External libraries and packages your project relies on
- Defined in manifest files: package.json, requirements.txt, pom.xml

GitHub Tools

- Dependabot — automatically checks for outdated or vulnerable dependencies
- Security alerts — notifies you when a dependency has a known CVE
- Lock files (package-lock.json) ensure reproducible builds

Repository Management

Section 5

Creating a Repository

From GitHub.com

- Click '+' → New repository → choose name, visibility, and options
- Initialize with README, .gitignore, and license
- Visibility options: Public (anyone can see) or Private (invited only)

From the Command Line

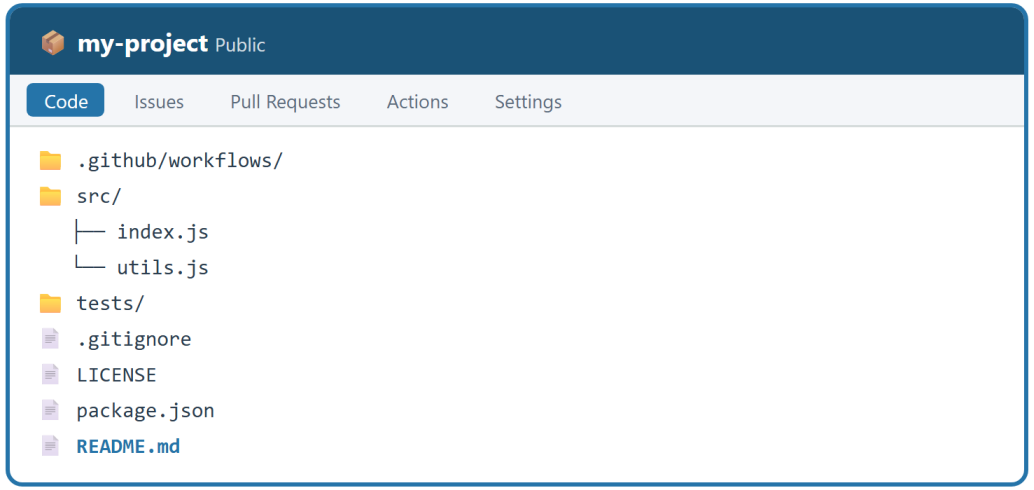
- Create locally with git init, then push to GitHub
- Or use GitHub CLI: gh repo create my-project --public

Creating & Connecting a Repository

```
# Create a new repository on GitHub.com, then:

# Option 1: Clone the empty repo
$ git clone https://github.com/username/my-project.git
$ cd my-project

# Option 2: Push an existing local project
$ cd my-existing-project
$ git remote add origin https://github.com/username/my-project.git
$ git push -u origin main

# Option 3: Use GitHub CLI
$ gh repo create my-project --public --clone
```

Repository Structure

Anatomy of a GitHub Repository



Branches

- `main` (default)
- `develop`
- `feature/login`
- `feature/api`

PR Workflow

1. Create branch
2. Make commits
3. Open Pull Request
4. Code review
5. Merge to main

Key Files
README.md · .gitignore · LICENSE

Essential Repository Files

README.md

- Project description, setup instructions, usage examples
- First thing visitors see — your project's front page

.gitignore

- Lists files and folders Git should not track (node_modules, .env, build/)
- GitHub provides templates for every language and framework

LICENSE

- Defines how others can use your code (MIT, Apache 2.0, GPL)

Repository Settings & Features

Access Control

- Add collaborators with Read, Write, or Admin permissions
- Use teams for organization-level access management

Branch Protection Rules

- Require pull request reviews before merging to main
- Require status checks (CI must pass) before merge
- Prevent force pushes to protected branches

Other Settings

- Enable/disable Issues, Wiki, Discussions, and GitHub Pages

Writing a Good README

````
# Project Name

A brief description of what this project does.

## Getting Started

### Prerequisites
- Node.js 20+
- npm or yarn

### Installation
```bash
git clone https://github.com/username/project.git
cd project
npm install
```

## Usage
```bash
npm start
```
````

Best Practices

Section 6

Repository Naming & Organization

Naming Conventions

- Use lowercase with hyphens: my-awesome-project (not MyAwesomeProject)
- Be descriptive: weather-api is better than project1
- Prefix with team or org name for clarity: team-frontend-dashboard

Repository Organization

- One repository per project or microservice
- Use consistent folder structure across team repos
- Keep repos focused — avoid monoliths with unrelated code

Documentation & Communication

Must-Have Documentation

- README.md — project overview, setup, and usage
- CONTRIBUTING.md — how to contribute, coding standards, PR process
- CHANGELOG.md — track notable changes across versions

Team Communication

- Write meaningful commit messages: 'Fix login timeout on SSO redirect' not 'fix bug'
- Use issue templates to standardize bug reports and feature requests
- Reference issues in PRs and commits (#42, Fixes #15)

Team Workflow Checklist

Before Starting Work

- Pull latest changes from main before creating a new branch
- Check open issues and PRs — avoid duplicate work

During Development

- Commit frequently with descriptive messages
- Push your branch regularly to back up your work

Before Merging

- Ensure all CI checks pass and code is reviewed
- Resolve all review comments before final merge

Key Takeaways

Key Takeaways

1. GitHub is a cloud platform built on Git that adds collaboration, CI/CD, and project management
2. Issues track work; Pull Requests enable reviewed, safe code integration
3. GitHub Actions automates build, test, and deploy workflows using YAML config
4. Workflows contain jobs with steps — use Marketplace actions for common tasks
5. Secrets keep credentials safe; environments add approval gates for production
6. Repositories should have a README, .gitignore, and clear naming conventions
7. Consistent workflows and documentation are the foundation of effective teamwork